BANKING LOAN RISK ANALYSIS APPLICATION

Technical Design Document

Python Full-Stack & Data Engineering Capstone

Design baseline: implemented application; architecture figure reserved for later revision.

Document	Details
System	Credit default / payment-risk analytics application
Architecture	Pandas ETL, MySQL, FastAPI, Streamlit
Risk approach	Transparent deterministic business rules (no AI / ML)
Status	Design document draft for review

1. Introduction and Objectives

Banks need a repeatable way to ingest credit records, validate their quality, analyze repayment behavior and surface transparent risk indicators. This project implements a local, modular banking analytics application around the UCI Default of Credit Card Clients dataset. Although the capstone title uses “loan risk,” the source is credit-card repayment data; the system does not originate or approve loans.

Business objectives

• Replace manual spreadsheet-only analysis with repeatable ETL, relational storage, REST APIs and interactive reporting.

• Identify customers for review using explainable payment-behavior, utilization and delinquency rules.

• Support portfolio analysis by risk band, demographics, credit exposure and historical monthly activity.

• Provide auditable incremental updates and role-based access to customer operations.

Scope

Included: historical dataset processing, validation, engineered features, customer and credit data storage, batch risk scoring, authenticated CRUD, portfolio analytics, CSV exports and automated tests. Excluded: machine-learning predictions, real-time banking feeds, automated credit decisions and production-grade deployment infrastructure.

2. Technology and Deployment

Layer	Implemented technology / purpose
Source	UCI Default of Credit Card Clients; Excel .xls
ETL	Python, Pandas; validation, transformations, full and incremental loading
Database	MySQL relational schema and indexes
Backend	FastAPI, Pydantic, MySQL Connector; REST/JSON
Security	Argon2 password hashing, JWT access tokens, RBAC
Frontend	Streamlit with Plotly charts and HTTP API client
Quality	Pytest, integration checks, structured logging

The application is designed for local execution. Streamlit calls FastAPI over HTTP; the API accesses MySQL. ETL and risk batch scripts access MySQL separately. The frontend does not require direct database credentials.

3. Logical System Design

Data processing path: UCI Excel source → extraction → data-quality validation → transformation and feature engineering → full or incremental relational loading → customer and credit tables. The incremental path also writes batch-audit information.

Risk assessment path: the implemented batch risk loader extracts and transforms source data, applies deterministic scoring rules, and persists risk results in MySQL. This is a batch computation, not an ML inference service.

Application path: user → Streamlit login/dashboard → HTTP requests → FastAPI authorization and services → MySQL → JSON/CSV response → Streamlit visualizations.

Architecture diagram: intentionally deferred pending approval of the revised PlantUML architecture. The flows above are the authoritative textual design for this draft.

4. ETL and Data Quality

Extraction and validation

The source contains 30,000 customer records with demographics, credit limits, six repayment-status observations, six bill amounts, six payment amounts and the historical next-month default outcome. Validation checks include missing and duplicate IDs, missing fields, invalid ranges and unexpected categorical codes. Undocumented education and marital-status values are retained as codes while labeled as “Others” for reporting.

Transformation and table preparation

Transformation creates normalized customer identifiers and labeled demographics, month-specific repayment/bill/payment fields, delay-month count, maximum delay status, six-month bill/payment aggregates, payment-delay flags and latest utilization ratio. The six monthly observations are unpivoted into monthly-credit-activity rows.

Initial load and reconciliation

The full loader inserts customer, credit-profile and monthly-activity records transactionally and checks table counts. For the original 30,000-record dataset, expected table counts are 30,000 customers, 30,000 credit profiles and 180,000 monthly activity rows.

Incremental processing

The incremental workflow compares incoming transformed records with stored state, classifies records as new, changed or unchanged, and loads only the necessary new/changed records. Batch audit tracks source counts, processed counts, status, timestamps and failures. Transaction rollback prevents partial updates on failure. The source-file based batch pipeline is not a streaming ingest system.

5. Relational Database Design

Table / group	Purpose
customers	Customer identity and demographic foreign keys
credit_profiles	Credit limit, engineered repayment summaries and historical default outcome
monthly_credit_activity	Six months of repayment status, billed amount and payment amount per customer
dim_gender / dim_education / dim_marital_status	Reference labels for demographic codes
customer_risk_assessment	Rule components, total score, band and rule version
etl_batch_audit	Incremental batch reconciliation and execution status
app_users	Application accounts, role and password hash

The customer identifier links the customer, credit profile, monthly activity and risk-assessment records. Reference dimensions normalize demographic labels. MySQL constraints and indexes support relational integrity and common lookup/filter workloads. The historical default outcome is retained for descriptive evaluation, not used as an input to the risk score.

6. Deterministic Risk Assessment

The system applies documented, interpretable business rules to payment behavior and credit exposure. Four component scores sum to a total of 0–11 points.

Component	Points	Interpretation
Delay frequency	0–3	Number of months with repayment delays
Delay severity	0–3	Worst observed repayment delay
Credit utilization	0–3	Exposure relative to credit limit
Payment behavior	0–2	Payments relative to billed amounts

Total score	Risk band
0–2	LOW
3–5	MEDIUM
6–11	HIGH

The risk results are review indicators rather than probabilities of default. The score is deterministic and explainable; it is not a statistical forecast or lending recommendation. Rule version is stored with assessments. For the original dataset, previously observed band counts were LOW 13,966; MEDIUM 9,673; HIGH 6,361.

7. Backend API Design

Route family	Primary capabilities
/health	Database connectivity and service health
/auth/login, /auth/me	JWT login and current-user identity
/customers, /customers/{id}	Paginated/filterable customer listing; detail; protected CRUD
/customers/{id}/activity, /risk	Monthly history and risk assessment for a customer
/analytics/summary	Portfolio-level key performance indicators
/analytics/risk-distribution	Risk band breakdown
/analytics/monthly-trends	Time-series credit activity metrics
/analytics/segments	Demographic segment comparisons
/export/customers	Authorized CSV customer export

FastAPI separates route handlers from application services and database access. List endpoints support pagination, filtering and sorting where implemented. HTTP error handling covers invalid inputs, missing records, authorization failures and database exceptions. Interactive API documentation is available through FastAPI’s built-in OpenAPI interface when the service is running.

8. Authentication, Authorization and Security

Users authenticate with stored Argon2 password hashes. On successful login, the backend issues a signed JWT with a configured expiry. Subsequent protected requests use a Bearer token; backend dependencies enforce access restrictions. Application users are stored in MySQL.

Role	Permissions
ADMIN	Read, customer create/update/delete, CSV export
ANALYST	Read, analytics and CSV export
VIEWER	Read-only dashboards and customer information

Environment variables hold database credentials and JWT settings; actual secrets should not be committed. Security checks are enforced by the backend rather than relying on frontend navigation alone. This design does not claim enterprise SSO, MFA, encryption-at-rest or deployment perimeter controls.

9. Frontend and User Workflows

Streamlit provides six functional views: Overview, Risk Analysis, Portfolio Analytics, Customer Explorer, Customer Management, and Reports & Export. Plotly supports interactive visualization. The API client sends authenticated HTTP requests, and the UI adapts available actions to the signed-in user role.

• **Overview:** portfolio summaries and headline KPIs.

• **Risk Analysis:** score bands and customer risk indicators.

• **Portfolio Analytics:** credit and payment trends, customer segments.

• **Customer Explorer:** search, filters, customer details and monthly history.

• **Customer Management:** role-protected create/update/delete operations.

• **Reports & Export:** downloadable filtered customer records for authorized roles.

10. Testing, Reliability and Observability

The completed regression suite was reported as **672 tests passed, four subtests passed, and one PytestCollectionWarning**. Testing covered backend/API behavior, ETL logic, frontend components and live integration workflows. Frontend statement coverage was reported as 83%. These results describe the tested local environment, not a formal security audit or production load test.

Additional verification included database connectivity, authentication/RBAC, data reconciliation, CRUD, reports/exports and end-to-end operation. Structured logs capture ETL, API and failure events. Incremental transaction handling and audit records support traceability.

11. Design Decisions and Trade-offs

Decision	Benefit	Trade-off
MySQL relational warehouse	Integrity, joins, familiar SQL	Requires schema and local DB setup
Rule-based scoring	Explainable, testable, no training	Thresholds are heuristic, not predictive

Decision	Benefit	Trade-off
Streamlit frontend	Rapid analytical UI development	Less UI customization than a full SPA
FastAPI service boundary	Clear API and auth enforcement	Additional local process to operate
File-based incremental ETL	Auditable, reproducible updates	Not real-time or automatically scheduled

12. Limitations and Future Work

• The source is a static historical credit-card dataset; its population and period may not reflect current banking portfolios.

• Risk bands depend on business-defined thresholds; they are not calibrated default probabilities.

• Batch risk refresh should be coordinated with customer changes and incremental ETL; operational scheduling is not claimed as implemented.

• Potential future work: approved architecture figure, more deployment automation, monitoring, database migration management, and additional data-quality dashboards.

13. Submission Mapping and References

This document fulfills the design-document portion of the non-AI banking capstone. The architecture diagram is a separate pending deliverable; code, README, API documentation, dataset details and presentation are separate artifacts.

Project specification: “Project 13 - Banking Loan Risk Analysis Application” (provided capstone brief).

Submission requirements: “Submission_Guidelines.pdf” (provided). The generic GenAI-specific wording in that document is not applicable to this explicitly non-AI capstone.

Dataset: UCI Machine Learning Repository, Default of Credit Card Clients, https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients

Implementation basis: project modules and milestone details discussed for the Banking Loan Risk Analysis Application. Review against the final repository before formal submission.